Lo storico prima dei modelli

Storico etichettato, piccoli modelli linguistici e cascate per classificare le richieste su hardware locale. Uno studio misurato

Giovanni Simonetti (ORCID 0009-0008-7543-6737)

Rapporto tecnico, bozza 0.2, ottobre 2026

Sommario

Le organizzazioni che vogliono classificare le richieste in arrivo con i modelli linguistici di solito partono dalla scelta del modello. Questo rapporto misura che cosa succede se partono invece dalle richieste che hanno già classificato. Su un carico pubblico di 4.500 richieste scritte da collaboratori retribuiti (crowdsourcing), da smistare verso 10 reparti, con 15.000 richieste passate già etichettate, eseguiamo in locale, su un PC da ufficio senza scheda grafica, quattro modelli a pesi aperti della stessa famiglia (da 0,5 a 7,6 miliardi di parametri) e li confrontiamo con due modi di usare lo storico: una ricerca deterministica e un classificatore di testo standard. Emergono tre risultati. Primo, il classificatore addestrato sullo storico risponde correttamente al 95,9% delle richieste (intervallo al 95% da 95,1% a 96,7%), contro l'86,6% del modello più grande da solo, con un costo troppo piccolo per essere confrontato. Secondo, le cascate di soli modelli, prima il piccolo e poi il grande quando il piccolo è incerto, costano dal 15% al 74% in più del modello grande da solo quando l'accuratezza deve restare entro mezzo punto dalla sua, in gran parte perché la fiducia dei modelli piccoli non separa abbastanza bene le loro risposte giuste da quelle sbagliate. Terzo, il modello linguistico si guadagna il suo posto dove lo storico è scarso: con meno di circa 100 esempi etichettati per categoria il modello da 7,6 miliardi da solo batte il classificatore, mentre un classificatore che gli passa le richieste su cui è incerto è accurato almeno quanto il modello da solo a ogni dimensione dello storico, gli manda il 57% delle richieste o meno da 32 esempi per categoria, ed è chiaramente più accurato da 64 (+2,7 punti, intervallo da +1,9 a +3,4). L'ordine che ne discende è catturare prima lo storico, usarlo con lo strumento più semplice che funziona, e tenere il modello linguistico per ciò che lo storico non copre. È un solo compito, una sola famiglia di modelli e una sola macchina, e indichiamo che cosa smentirebbe il risultato. Codice, risposte salvate dei modelli e ogni numero sono pubblici.

1. Introduzione

Classificare le richieste in arrivo è uno degli usi più comuni dei modelli linguistici nei servizi: un ticket, un'email o un messaggio in chat arriva e deve raggiungere il gruppo giusto. La domanda abituale è quale modello usare, e come spendere meno. Le cascate di modelli rispondono alla seconda parte: un modello piccolo risponde per primo e passa la richiesta a uno più grande solo quando la sua fiducia tarata è troppo bassa [1, 2, 3].

La maggior parte delle organizzazioni che classificano richieste, però, possiede già qualcosa di più prezioso di qualunque modello: migliaia di richieste passate con la categoria che una persona ha assegnato loro. Questo rapporto si chiede quanto vale quello storico, misurato su modelli reali eseguiti su hardware comune, e in che ordine storico e modelli vadano usati.

Una bozza precedente di questo lavoro [15] studiava in simulazione quando una cascata dovrebbe avere un terzo stadio, e proponeva una regola pratica: quando i modelli vicini distano meno di 5x nel costo, il terzo stadio dovrebbe essere fatto di regole, non di un modello intermedio. Eseguire modelli reali ha cambiato la domanda. Uno stadio basato sullo storico etichettato ha fatto più di qualunque modello, e un semplice classificatore addestrato sullo stesso storico ha fatto ancora di più. Questo rapporto presenta quelle misure.

I contributi sono:

- **Un confronto misurato** fra quattro modelli locali, le loro cascate, una ricerca deterministica nello

storico e un classificatore di testo addestrato su di esso, sulle stesse richieste, con la stessa divisione dei dati e la stessa misura di costo.
- **Una curva di quanto storico serve** perché il classificatore superi un modello linguistico che non vede esempi, e di che cosa ottiene nel frattempo un classificatore che passa al modello le richieste incerte.
- **Una prima verifica su modelli reali della regola del 5x** di [15], nell'intervallo di costi coperto dal test.
- **Tutto il necessario per ripeterlo**: codice, le risposte salvate di ogni modello e il protocollo per eseguirlo su altri modelli e carichi.

Nessuno dei componenti è nuovo. Cascate, fiducia tarata, ricerca dei vicini più simili e classificatori lineari di testo sono tutti consolidati. Il rapporto aggiunge la misura, su hardware che una piccola organizzazione possiede, e l'ordine che ne discende.

2. Lavori correlati

Cascate e instradamento. FrugalGPT [1] mette in catena API di modelli e si ferma alla prima risposta accettabile; AutoMix [7] usa l'autoverifica per decidere quando salire. UCCI [2] tara la fiducia del modello piccolo con la regressione isotonica e riporta una riduzione dei costi del 31% con due modelli serviti in locale; Conformal Cascade [3] usa la previsione conforme e riporta il 43%, con garanzie formali di accuratezza. CARGO [4] e i token di confidenza [5] studiano più in generale l'instradamento guidato dalla fiducia, e [6] esamina quando i modelli sanno di non sapere. Questi lavori confrontano modelli con modelli. Qui fa parte del confronto anche uno stadio che non usa alcun modello.

Modelli linguistici e classificatori supervisionati. Che un classificatore supervisionato addestrato su abbastanza dati etichettati possa eguagliare o battere un grande modello interrogato senza esempi è noto dalla letteratura sull'apprendimento da pochi esempi [8, 9]. SetFit [9], per esempio, raggiunge un'accuratezza elevata da pochi esempi etichettati per classe con modelli piccoli. Il rapporto non lo presenta come una scoperta; misura dove cade il punto di incrocio su un carico, con modelli locali e i loro costi reali.

Dati e modelli. Il carico è il livello dei domini di CLINC150 [10]; i modelli sono Qwen2.5 [11], serviti da Ollama; il classificatore è la regressione logistica di scikit-learn [12] su caratteristiche TF-IDF. La taratura segue la pratica corrente [13]; il test di taratura è quello di Hosmer e Lemeshow [14].

3. Impostazione

3.1 Il carico

CLINC150 [10] contiene richieste a un assistente virtuale scritte da collaboratori retribuiti, ciascuna etichettata con uno di 150 intenti raggruppati in 10 domini, come banca, viaggi o lavoro. Usiamo i domini come reparti: ogni richiesta va assegnata a uno dei 10. Lo storico etichettato è l'insieme di addestramento, 15.000 richieste; l'insieme di prova ne ha 4.500. Assegnare una richiesta a uno di un elenco chiuso di reparti ha la stessa struttura dello smistamento di un ticket, ma le richieste non sono ticket, e la sezione 6 torna su questo punto.

Le 4.500 richieste di prova sono divise una volta, con un seme fisso, in una metà di taratura e una metà di test da 2.250 ciascuna. Ogni soglia è scelta sulla metà di taratura; ogni accuratezza è misurata sulla metà di test, con intervalli bootstrap al 95% su 2.000 ricampionamenti della metà di test. Gli intervalli tengono fisse le soglie: non includono la variabilità della divisione o della scelta della soglia. Le richieste fuori ambito, che CLINC150 fornisce, non sono usate.

3.2 I modelli

Quattro modelli a pesi aperti della famiglia Qwen2.5 [11], da 0,49, 1,54, 3,09 e 7,61 miliardi di parametri, nelle versioni quantizzate predefinite distribuite da Ollama, sono eseguiti in locale su un PC da ufficio (Intel Core

Ultra 7 265, 32 GB di memoria, nessuna scheda grafica). Ogni modello vede l'elenco dei 10 reparti, ciascuno descritto dagli argomenti che tratta (i nomi dei suoi 15 intenti in CLINC150), nessuna richiesta di esempio, e risponde con il nome di un reparto. La decodifica è greedy. Una risposta che nomina uno degli argomenti di un reparto invece del reparto conta come quel reparto, e i piccoli errori di scrittura sono ricondotti al nome più vicino; le risposte che non corrispondono a nulla contano come sbagliate. Il segnale di fiducia è la probabilità che il modello assegna alla propria risposta, il prodotto delle probabilità dei suoi token. Il costo è il tempo di calcolo per richiesta riportato dal server (lettura della richiesta e generazione della risposta; il caricamento del modello è escluso), con le istruzioni tenute in memoria fra una richiesta e l'altra, come farebbe un server in produzione. Il costo di un modello è la sua media su tutte le 4.500 richieste, e il costo di una cascata è calcolato da queste medie.

3.3 Due modi di usare lo storico

Ricerca. Ogni richiesta è confrontata con ogni richiesta passata tramite la similarità coseno TF-IDF su parole e coppie di parole. Se le cinque richieste passate più simili hanno tutte lo stesso reparto, e la più vicina è abbastanza simile, la ricerca risponde con quel reparto; altrimenti passa la richiesta. La soglia di similarità è scelta sulla metà di taratura, come parte di ogni cascata (sezione 3.4).

Classificatore. Una regressione logistica su caratteristiche TF-IDF di parole e coppie di parole (frequenza dei termini sublineare, regolarizzazione $C = 10$, per il resto le impostazioni predefinite di scikit-learn), addestrata sullo storico etichettato, senza altre regolazioni. Risponde sempre.

Classificatore, poi modello. Il classificatore risponde quando la sua probabilità più alta raggiunge una soglia, altrimenti risponde il modello da 7,6 miliardi. La soglia (su una griglia di passo 0,02, applicata alla probabilità grezza) è quella con l'accuratezza più alta sulla metà di taratura; il costo non entra nella scelta. Il suo calcolo è all'incirca la quota di richieste che passa al modello.

3.4 Le cascate

Gli stadi lavorano in sequenza; la risposta di uno stadio è accettata quando la sua fiducia tarata raggiunge una soglia, altrimenti la richiesta passa oltre, e l'ultimo stadio risponde sempre. La fiducia è tarata sulla metà di taratura per intervalli di quantili. La soglia è quella con il costo più basso sulla metà di taratura fra quelle che tengono l'accuratezza entro mezzo punto dal modello più grande da solo. Quando una cascata comincia con la ricerca, la sua soglia di similarità è scelta insieme alla soglia di fiducia. Il risparmio è uno meno il rapporto fra il costo della cascata e quello del modello più grande da solo.

4. Risultati

4.1 I modelli da soli

Tabella 1: Ogni modello da solo, senza richieste di esempio. Risposte giuste sulla metà di test; secondi per richiesta in media su tutte le 4.500 richieste.

Modello	Risposte giuste	Secondi per richiesta	Separazione fra risposte giuste e sbagliate (AUROC)
0,5 mld	43,1%	0,088	0,73
1,5 mld	72,4%	0,171	0,71
3,1 mld	69,9%	0,296	0,74
7,6 mld	86,6% (da 85,2% a 88,0%)	0,591	0,77

L'accuratezza cresce con la dimensione, con un'inversione fra 1,5 e 3,1 miliardi. I segnali di fiducia sono deboli: un AUROC fra 0,71 e 0,77 significa che la probabilità che un modello assegna alla propria risposta separa solo moderatamente le risposte giuste da quelle sbagliate. La simulazione di [15] aveva ipotizzato segnali fra circa 0,86 e 0,96.

4.2 Le cascate di soli modelli costano di più

Ogni cascata fatta solo di modelli, da due a quattro stadi, costa più del modello da 7,6 miliardi da solo, con accuratezza uguale o di poco inferiore: dal 15% in più (0,5 miliardi, poi 7,6) al 74% in più (tutti e quattro). Con segnali così deboli, una soglia che tiene l'accuratezza entro mezzo punto manda comunque quasi ogni richiesta al modello grande, e il calcolo dei modelli piccoli si aggiunge. Con 1,5 miliardi e poi 7,6, il modello piccolo trattiene solo il 5% delle richieste; con 0,5 e poi 7,6, nessuna.

È l'opposto dei risparmi del 31% e del 43% riportati da [2, 3] su altri compiti e modelli. Compiti, modelli e tolleranze sono diversi, ma una differenza probabile è il segnale di fiducia: una cascata risparmia solo se il modello piccolo sa quando ha ragione.

4.3 Lo storico da solo

Con una similarità di 0,3, il valore che ogni cascata con la ricerca ha scelto sulla metà di taratura (e il valore più basso della griglia, quindi valori inferiori non sono stati provati), la ricerca risponde al 63% delle richieste di test e ne azzecca il 98,7%, in circa un terzo di millisecondo per richiesta. Alzando la soglia di similarità la copertura scende (59% a 0,4, 35% a 0,6, 20% a 0,7) mentre l'accuratezza resta vicina al 99%.

Il classificatore risponde a tutte:

Tabella 2: Lo storico usato da un classificatore, a confronto con il modello più grande.

Approccio	Risposte giuste	Calcolo per richiesta
Classificatore su 15.000 richieste etichettate	**95,9%** (da 95,1% a 96,7%)	circa 0,01 ms
Modello da 7,6 mld da solo, senza esempi	86,6% (da 85,2% a 88,0%)	591 ms

Il tempo del classificatore è una previsione in blocco misurata su una macchina diversa da quella dei modelli, che lavoravano una richiesta alla volta; è solo un ordine di grandezza, decine di migliaia di volte più piccolo di quello del modello da 7,6 miliardi in ogni caso. Su questo compito lo storico etichettato, usato da un classificatore standard, batte il modello più grande di oltre 9 punti.

4.4 Lo storico davanti ai modelli

Messa prima dei modelli, la ricerca cambia l'economia di ogni cascata (tabella 3).

Tabella 3: Cascate con e senza la ricerca, sulla metà di test.

Configurazione	Calcolo risparmiato rispetto a 7,6 mld da solo	Differenza di accuratezza, punti	Quota a cui risponde ogni stadio
ricerca, 1,5 mld, 7,6 mld	89% (da 89% a 90%)	+0,5 (da -1,2 a +2,1)	63% / 37% / 0%
ricerca, 7,6 mld	63% (da 61% a 65%)	+5,2 (da +4,4 a +6,3)	63% / 37%

Configurazione	Calcolo risparmiato rispetto a 7,6 mld da solo	Differenza di accuratezza, punti	Quota a cui risponde ogni stadio
1,5 mld, 7,6 mld (senza storico)	-24% (da -25% a -23%)	-0,1 (da -0,4 a +0,1)	5% / 95%

La ricerca seguita dal modello da 1,5 miliardi pareggia il modello da 7,6 miliardi da solo con circa un nono del suo calcolo; in questa configurazione il modello da 7,6 miliardi non riceve alcuna richiesta. La ricerca seguita dal modello da 7,6 miliardi è più accurata di 5 punti del modello da solo e risparmia il 63% del suo calcolo. Su questo compito entrambe sono superate dal classificatore della sezione 4.3, più accurato di tutte e due con un costo trascurabile.

4.5 Quanto storico serve

Lo storico non è sempre grande. La figura 1 mostra che cosa succede quando sono disponibili solo pochi esempi etichettati per reparto, estratti a caso dai 15.000 (cinque estrazioni per dimensione).

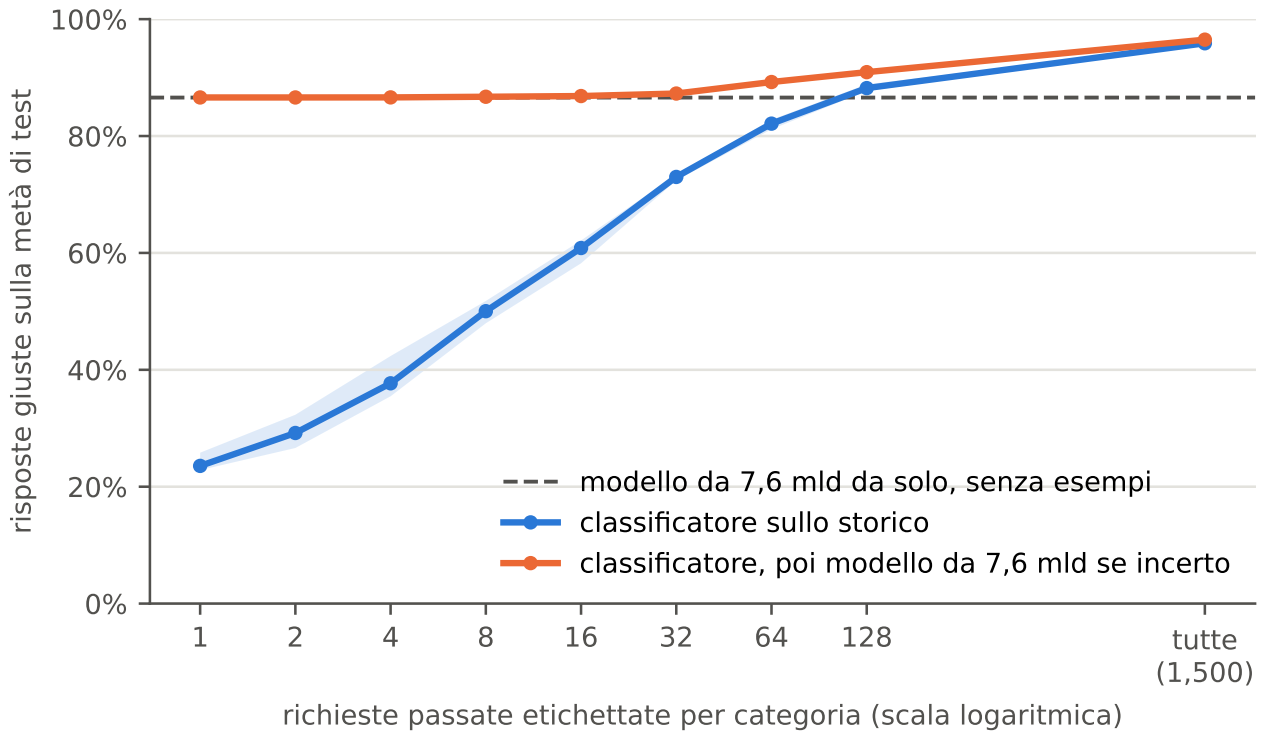


Figura 1: Risposte giuste sulla metà di test al crescere dello storico etichettato. Blu: il classificatore da solo (fascia: intervallo sulle cinque estrazioni). Arancione: il classificatore, che passa al modello da 7,6 miliardi le richieste su cui non è abbastanza sicuro, con la soglia scelta sulla metà di taratura. Tratteggio: il modello da 7,6 miliardi da solo, senza esempi.

Tabella 4: Il classificatore e la combinazione classificatore-modello al crescere dello storico (media di cinque estrazioni; intervalli bootstrap appaiati). Il modello da 7,6 miliardi da solo ottiene l'86,6%.

Esempi etichettati per reparto	Classificatore da solo	Classificatore, poi 7,6 mld	Rispetto a 7,6 mld da solo, punti	Quota inviata a 7,6 mld
8	50,0%	86,7%	+0,1 (da +0,0 a +0,3)	94%
32	73,0%	87,3%	+0,7 (da +0,1 a +1,3)	57%
64	82,1%	89,2%	+2,7 (da +1,9 a +3,4)	39%
128	88,2%	90,9%	+4,4 (da +3,3 a +5,3)	22%
1.500 (tutti)	95,9%	96,5%	+9,9 (da +8,5 a +11,3)	2%

Emergono tre regimi:

- **Poco storico** (fino a circa 16 esempi per reparto): il modello linguistico porta quasi tutto il peso. Legge le descrizioni dei reparti e non ha bisogno di esempi; il classificatore non può.
- **Un po' di storico** (da circa 32 a 128): la combinazione è la soluzione migliore. Con 64 esempi per reparto è più accurata di 2,7 punti del modello da 7,6 miliardi da solo e gli manda solo il 39% delle richieste; con 128, di 4,4 punti mandandogliene il 22%.
- **Storico ampio** (oltre circa 100 per reparto): il classificatore da solo supera il modello linguistico. Il modello aggiunge ancora qualcosa alla combinazione con 128 esempi (+2,7 punti sul classificatore da solo), e poco con lo storico completo (+0,6).

4.6 La regola del 5x su modelli reali

La simulazione di [15] prevedeva che, con modelli vicini distanti meno di 5x nel costo, un modello intermedio non convenisse e uno stadio di regole sì. Qui ogni terna di modelli è sotto 5x (il passo medio va da 1,8x a 2,6x). Delle quattro terne, aggiungere un modello intermedio ha perso chiaramente in tre e guadagnato chiaramente in una (0,5, 1,5 e 3,1 miliardi, di 48 punti); aggiungere la ricerca ha guadagnato chiaramente in tutte e quattro, da 89 a 113 punti di risparmio ("chiaramente": l'intero intervallo al 95% da una parte dello zero). Questi guadagni sono misurati rispetto a cascate a due stadi che già perdono, e le quattro terne condividono modelli, quindi non sono prove indipendenti. Nel suo intervallo inferiore la regola ha retto in tre casi su quattro per il modello intermedio e in tutti e quattro per la ricerca. Il suo intervallo superiore, da 5x in su, non è messo alla prova da questi modelli. La correzione più importante riguarda la simulazione stessa: uno stadio di ricerca reale ha coperto il 63% delle richieste, non il 10-30% ipotizzato, e i segnali di fiducia reali sono stati più deboli di qualunque segnale simulato.

4.7 Diagnostica

Gli errori dei modelli sono poco correlati (da 0,12 a 0,31), quindi in linea di principio passare la richiesta al modello grande può aiutare: il modello grande spesso azzecca ciò che il piccolo sbaglia. La difficoltà è sapere quando farlo. Sulla metà di test la fiducia tarata del modello da 7,6 miliardi non supera il test di taratura al livello del 5% (chi quadro 12,1 con 4 gradi di libertà, $p = 0{,}016$), anche se lo supererebbe correggendo per quattro test; gli altri tre lo superano.

5. Che cosa suggerisce

Per un'organizzazione che classifica richieste e ha, o può costruire, uno storico di casi classificati, le misure suggeriscono un ordine:

1. **Catturare prima lo storico.** I casi passati etichettati hanno valso più di qualunque modello provato. Il loro valore dipende da quanti sono, da quanto bene sono etichettati e da quanto sono recenti.
2. **Usarlo con lo strumento più semplice che funziona.** Una ricerca che risponde solo quando il passato è concorde ha quasi sempre ragione su ciò a cui risponde; un classificatore standard risponde a tutto e, con abbastanza storico, batte ogni modello provato.
3. **Tenere il modello linguistico per ciò che lo storico non copre**: reparti con pochi casi passati, come misurato qui, e, come ipotesi che questo test non ha coperto, categorie nuove e richieste diverse da tutto ciò che si è visto. Lì la capacità del modello di leggere una descrizione, senza esempi, è ciò che manca al classificatore.
4. **Non contare su una cascata di modelli per risparmiare**, a meno che non sia dimostrato, sui propri dati, che la fiducia del modello piccolo separa le sue risposte giuste da quelle sbagliate.

Lo stesso storico rende anche il modello verificabile: con gli esiti passati si possono misurare taratura e deriva, e scegliere una configurazione sui numeri invece che sulla dimensione del modello.

6. Limiti

- **Un solo compito.** Dieci reparti ampi con 1.500 esempi etichettati ciascuno sono un caso facile per un classificatore. Le code di ticket reali hanno più categorie, volumi disomogenei, etichette più rumorose e argomenti che cambiano; il punto di incrocio della sezione 4.5 può spostarsi in entrambe le direzioni.
- **Le richieste non sono ticket.** Sono brevi richieste a un assistente, scritte da collaboratori retribuiti, con la struttura dello smistamento ma non il suo materiale. Le parafrasi ravvicinate fra storico e richieste di test favoriscono la ricerca e il classificatore.
- **Una famiglia di modelli, una macchina, nessun esempio nel prompt.** Modelli più grandi o diversi, altre quantizzazioni, o prompt con esempi presi dallo storico potrebbero cambiare i risultati dei modelli; nessun modello commerciale di frontiera è stato provato.
- **Nulla è stato ottimizzato.** Né il classificatore né il prompt dei modelli sono stati regolati. Regolare l'uno o l'altro potrebbe spostare il divario della sezione 4.3 in entrambe le direzioni.
- **Costi.** I costi dei modelli sono tempo di calcolo su una CPU; il classificatore è stato cronometrato su un'altra macchina, in blocco, ed è dato come ordine di grandezza. Elettricità, hardware e manutenzione non sono inclusi.
- **Storico casuale.** Gli storici ridotti della sezione 4.5 sono campioni casuali. Lo storico di un'organizzazione reale non lo è: sovrarappresenta le richieste frequenti ed è in ritardo su quelle nuove.

Che cosa smentirebbe la conclusione: un carico realistico in cui un classificatore addestrato sullo storico disponibile non batte un modello linguistico che non vede esempi; oppure uno in cui un modello linguistico con esempi presi dallo storico batte sia il classificatore sia la combinazione a costo comparabile.

7. Prossimi passi

Il protocollo è pronto per tre estensioni: gli stessi modelli con esempi dallo storico nel prompt; una seconda famiglia di modelli; e un carico più difficile, con molte categorie e pochi esempi ciascuna (Banking77, 77 categorie, è già supportato dal codice). Il contributo più utile sarebbe uno reale: uno storico di ticket di assistenza, e che cosa ottiene ciascun approccio su di esso.

Riproducibilità

Codice, la risposta salvata di ogni modello a ogni richiesta e questo rapporto in italiano e in inglese sono su https://github.com/gionnysimonetti-dev/calibrated-funnel con licenza MIT, archiviati su Zenodo. Dalle risposte salvate, `python -m realtest.analyze --task departments --shots 0` scrive `realtest/REPORT_departments.md` (costi, AUROC e ricerca nelle sezioni 4.1 e 4.3; sezioni 4.2, 4.4, 4.6, 4.7), e `python -m realtest.baselines --task departments` scrive `realtest/BASELINES_departments.md` (accuratezze con intervalli nelle sezioni 4.1 e 4.3; sezione 4.5) e i dati della figura 1, che `python paper/make_figure_history.py` disegna. Rieseguire i modelli richiede Ollama e circa due ore sul PC descritto nella sezione 3.2: `python -m realtest.run_models --task departments --shots 0`.

Uso dell'intelligenza artificiale

Il codice e parte dell'analisi sono stati sviluppati con un assistente AI (Claude, Anthropic), che ha anche contribuito alla stesura di questo rapporto. L'autore ha progettato lo studio, eseguito i modelli, verificato i risultati e si assume la responsabilità del contenuto.

Bibliografia

1. L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
2. V. Kotte. UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. arXiv:2605.18796, 2026.
3. Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference. arXiv:2607.25018, 2026.
4. A. Barrak, Y. Fourati, M. Olchawa, E. Ksontini, K. Zoghlami. CARGO: A Framework for Confidence-Aware Routing of Large Language Models. arXiv:2509.14899, 2025.
5. Y.-N. Chuang, P. K. Sarma, P. Gopalan, J. Boccio, S. Bolouki, X. Hu, H. Zhou. Learning to Route LLMs with Confidence Tokens. arXiv:2410.13284, 2024.
6. C. Hao, W. Lu, Y. Ishiwaka, Z. Li, W. Wan, Y. Chen. When Models Know When They Do Not Know: Calibration, Cascading, and Cleaning. arXiv:2601.07965, 2026.
7. P. Aggarwal, A. Madaan, et al. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963, 2023.
8. T. Brown et al. Language Models are Few-Shot Learners. NeurIPS, 2020.
9. L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, O. Pereg. Efficient Few-Shot Learning Without Prompts. arXiv:2209.11055, 2022.
10. S. Larson et al. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. EMNLP-IJCNLP, 2019.
11. Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
12. F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2011.
13. C. Guo et al. On Calibration of Modern Neural Networks. ICML, 2017.
14. D. W. Hosmer, S. Lemeshow. Goodness-of-fit tests for the multiple logistic regression model. Communications in Statistics, 1980.
15. G. Simonetti. La regola del 5x: come scegliere il terzo stadio di una cascata di modelli governata dalla fiducia. Rapporto tecnico, bozza 0.1, ottobre 2026. Nello stesso repository, paper/five-x-rule.it.md.